

Cours 6

Programmation orientée objet

Jérémy Kairis

L'héritage

L'héritage

:

C# prend en charge l'héritage simple, ce qui signifie qu'une classe ne peut hériter que d'une et d'une seule autre classe.

La nouvelle classe (dérivée) hérite ensuite tous les membres non privés de la classe de base en plus de tous ses propres membres qu'elle définit pour elle-même.

L'héritage se définit pas le deux points “:”.


```
class Personne
{
    public int Age { get; set; }
}
```

```
class Etudiant : Personne
{
    public DateTime InscritLe { get; set; }
}
```

L'héritage

Object

Lorsque qu'aucun héritage n'est défini, la classe héritera cependant de la classe « ***System.Object*** », appelée en C# la classe de base universelle.

Ce qui implique que la classe « ***System.Object*** » est la mère de tout élément et que par conséquent nous obtenons toujours les méthodes non privées qu'elle définit comme “***Equals***” et “***ToString***”.

```
Console.WriteLine(new Exemple());
```

```
class Exemple
```

```
{
```

```
    public override string ToString()
```

```
        => "Ceci est un exemple.";
```

```
}
```


L'héritage

virtual - override

Que nous voulions redéfinir ou dissimuler une méthode nous voulons, dans les deux cas, modifier le comportement de notre méthode par rapport à la classe de base.

Dans le cadre de la redéfinition, nous utiliserons les mots clés « ***virtual*** » et « ***override*** ».

Le mot-clé « ***virtual*** » va signaler que la méthode de la classe de base est redéfinissable.

Pour redéfinir la méthode dans la classe dérivée, nous emploierons le mot-clé « ***override*** ».

```
class Personne
{
    public virtual void SePresenter()
    {
        Console.WriteLine("Je suis une personne.");
    }
}
```

```
class Etudiant : Personne
{
    public override void SePresenter()
    {
        Console.WriteLine("Je suis un étudiant.");
    }
}
```


L'héritage

base

Le mot clé « ***base*** » sert à accéder aux membres de la classe de base à partir d'une classe dérivée.

```
class Personne
{
    public virtual void SePresenter()
    {
        Console.WriteLine("Je suis une personne.");
    }
}
```

```
class Etudiant : Personne
{
    public override void SePresenter()
    {
        // Appel la version de la méthode de la classe de base
        base.SePresenter();
        Console.WriteLine("Je suis un étudiant.");
    }
}
```

L'héritage

protected

Le modificateur d'accès ***protected*** est utilisé pour déclarer des membres de classe (champs, propriétés, méthodes) qui sont accessibles uniquement à l'intérieur de la classe et à partir des classes dérivées.


```
class Vehicule
{
    public string Marque { get; set; }

    // Méthode protégée pour démarrer le moteur
    protected void DemarrerMoteur()
    {
        Console.WriteLine($"Le moteur du véhicule démarre.");
    }
}

class Voiture : Vehicule
{
    public void DemarrerVoiture()
    {
        // Utilisation de la méthode protégée de la classe de base
        DemarrerMoteur();
        Console.WriteLine($"La voiture démarre.");
    }
}
```

L'héritage

sealed

Si pour quelque raison que ce soit nous souhaitons empêcher notre classe d'être héritée par une classe dérivée, nous pouvons utiliser le mot clé « ***sealed*** ».

Comme par exemple la classe ***String*** afin de garantir le cohérence de manipulation des chaînes de caractères.

Nous pouvons également utiliser le modificateur « ***sealed*** » sur une méthode ou une propriété qui substitue une méthode ou une propriété virtuelle dans une classe de base.

```
sealed class CalculateurTaxe
{
    const double TauxStandard = 0.21;

    public double CalculerTaxe(double montant)
    {
        return montant * TauxStandard;
    }
}
```


Exercice

Banque

6. Créer une classe « SavingsAccount » pour la gestion d'un carnet d'épargne implémentant :

- Les propriétés publiques:
 - string Number
 - double Balance (lecture seule)
 - DateTime DateLastWithdraw
 - Person Owner
- Les méthodes publiques :
 - void Withdraw(double amount)
 - void Deposit(double amount)

Exercice

Banque

7. Définir la classe « Account » reprenant les parties commune aux classes « CurrentAccount » et « SavingsAccount » en utilisant les concepts d'héritage, de redéfinition de méthodes et si besoin, de surcharge de méthodes et d'encapsulation.

Attention le niveau d'accessibilité du mutateur de la propriété Balance doit rester « private ».

8. Modifier la classe « Bank » afin qu'elle ne travaille qu'avec des comptes.

Les classes abstraites

Les classes abstraites

Introduction

Le but de l'héritage est de définir les fonctionnalités communes à toutes les classes qui en sont dérivées afin de leur éviter de devoir redéfinir les éléments communs.

Cependant, il arrive que cette fonctionnalité commune ne soit pas suffisamment complète pour faire un objet utilisable et que l'on retrouve des fonctionnalités manquantes dans chaque classe dérivée.

Dans ce cas, nous pouvons spécifier que la classe ne peut être utilisée qu'à des fins d'héritage et non pas à l'instanciation d'objets.

Pour ce faire, nous utiliserons le mot clé « ***abstract*** » dans la déclaration de classe.

Nous obtenons au final une classe abstraite.

Les classes abstraites

abstract

En dehors des classes, le mot-clé « ***abstract*** » peut être utilisé avec des méthodes, des propriétés, des indexeurs ou des événements.

S'il s'agit de méthodes abstraites, nous ne devons spécifier que le prototype de la méthode suivi d'un point-virgule.

Les membres marqués comme abstraits, inclus dans une classe abstraite, doivent être implémentés par les classes qui héritent de la classe abstraite à l'aide du mot-clé « ***override*** ».


```
abstract class Forme
{
    public abstract void Dessiner();
}

class Cercle : Forme
{
    public override void Dessiner() => Console.WriteLine("Dessiner un cercle");
}

class Rectangle : Forme
{
    public override void Dessiner() => Console.WriteLine("Dessiner un rectangle");
}
```

Exercice

Banque

9. Définir la classe « Account » comme étant abstraite.
10. Ajouter une méthode abstraite « protected » à la classe « Account » ayant le prototype « double CalculInterets() » en sachant que pour un livret d'épargne le taux est toujours de 4.5% tandis que pour le compte courant si le solde est positif le taux sera de 3% sinon de 9.75%.
11. Ajouter une méthode « public » à la classe « Account » appelée « ApplyInterest » qui additionnera le solde avec le retour de la méthode « CalculInterest ».

Le polymorphisme

Le polymorphisme

Exercice

Créer un programme qui utilise le polymorphisme avec différents types de véhicules ayant chacun son propre mode de déplacement, tout en utilisant une méthode commune pour les faire se déplacer.

Le polymorphisme

Définition

Par héritage, une classe peut être utilisée comme plusieurs types.

Elle peut être utilisée comme son propre type, tout type de base ou tout type d'interface (si elle implémente des interfaces).

C'est ce qu'on appelle le polymorphisme.

Le polymorphisme

Conversions

Par exemple, nous avons une classe de base ***Animal*** avec une méthode ***Parler***.

Deux classes dérivées, ***Chien*** et ***Chat***, héritent de la classe ***Animal*** et redéfinissent la méthode ***Parler*** pour émettre des sons spécifiques.

Lorsque nousinstancions des objets de ces classes dérivées et appelons la méthode ***Parler***, le polymorphisme fait en sorte que la méthode appropriée soit appelée en fonction du type de l'objet. Ainsi, même si les deux objets sont de type ***Animal***, leurs méthodes ***Parler*** renvoient le son spécifique à leur classe dérivée.

```
Animal animal1 = new Chien();  
Animal animal2 = new Chat();
```

```
animal1.Parler(); // Appelle la méthode Parler de la classe Chien  
animal2.Parler(); // Appelle la méthode Parler de la classe Chat
```

```
abstract class Animal  
{  
    public abstract void Parler();  
}  
class Chien : Animal  
{  
    public override void Parler() => Console.WriteLine($"Le chien aboie : Woof! Woof!");  
}  
class Chat : Animal  
{  
    public override void Parler() => Console.WriteLine($"Le chat miaule : Miaou!");  
}
```

Exercice

Banque

Créer une “Pull request” sur GitHub pour intégrer vos développements avec le projet Bank-2025 original.